

Tweet Sentiment Analysis

Zhen Guo (CS6120) and Vikram Chowdhary (CS4120)

Introduction

In today's world, Twitter provides people with a way to publicly express their thoughts on any given subject in a concise, condensed format. This allows us to use tweets as a way to predict users' thoughts or feelings on a certain subject. When this information is considered alongside the fact that Twitter has approximately 450 million monthly active users, it becomes evident that natural language processing and machine learning models that can classify the sentiments expressed in tweets can be powerful tools to understand the emotions being expressed by a significant portion of the world's population [2].

Being equipped with this information, our goal for this project was to compare and contrast the performance of different machine learning models on the task of classifying the sentiment of a tweet as positive or negative. To do so, we began with a dataset from Kaggle consisting of over 27,000 tweets, and generated word embeddings for each tweet using sentence-transformer's paraphrase-distilroberta-base-v2. We then trained a Logistic Regression classifier and a Neural Net on this data, and evaluated each model's performance using standardized metrics such as accuracy, precision, recall, and F1 score.

Implementation

Data Overview

As previously mentioned, the dataset we used for this project was found on Kaggle and consisted of over 27,000 tweets. The original dataset can be found at [this link](#) [3]. For each tweet, the feature 'textID' contains a unique identifier, 'text' contains the original text of the tweet, 'selected_text' stores selected text from the tweet, and 'sentiment' labels the sentiment of the tweet as either 'positive', 'negative', or 'neutral'. As shown in the appendix, the number of tweets that are labeled as 'positive' is 8582 and the number for 'negative' tweets is 7781. Therefore, the dataset has a relatively balanced number of positive and negative tweets.

We removed all tweets from the dataset where the original text was not a string, as there was only one instance of this, we reasoned that it would be more time-effective to remove the sample from the dataset rather than parsing it to a string. We also removed all tweets that were classified as 'neutral', as our original idea for this project was to perform a binary classification of the data as either positive or negative sentiment. Before training the models, we removed the textID and selected_text features, and converted the text of the Tweets to embeddings. The number of tweets for each class after cleaning is shown in fig 1. After completing the data cleaning process, we partitioned the final dataset into testing and training portions with a 70/30 split using sklearn's train_test_split function.

We initially tried to generate embeddings for each tweet using BERTweet at Professor Amir's suggestion, however we were not able to find enough documentation about this model to be confident that we were using it correctly. Instead, we decided to use the

[paraphrase-distilroberta-base-v2](#) model from Sentence-Transformers, as we have had experience using this model and package before so we could be confident of our methodology and results while using it to generate our embeddings [4].

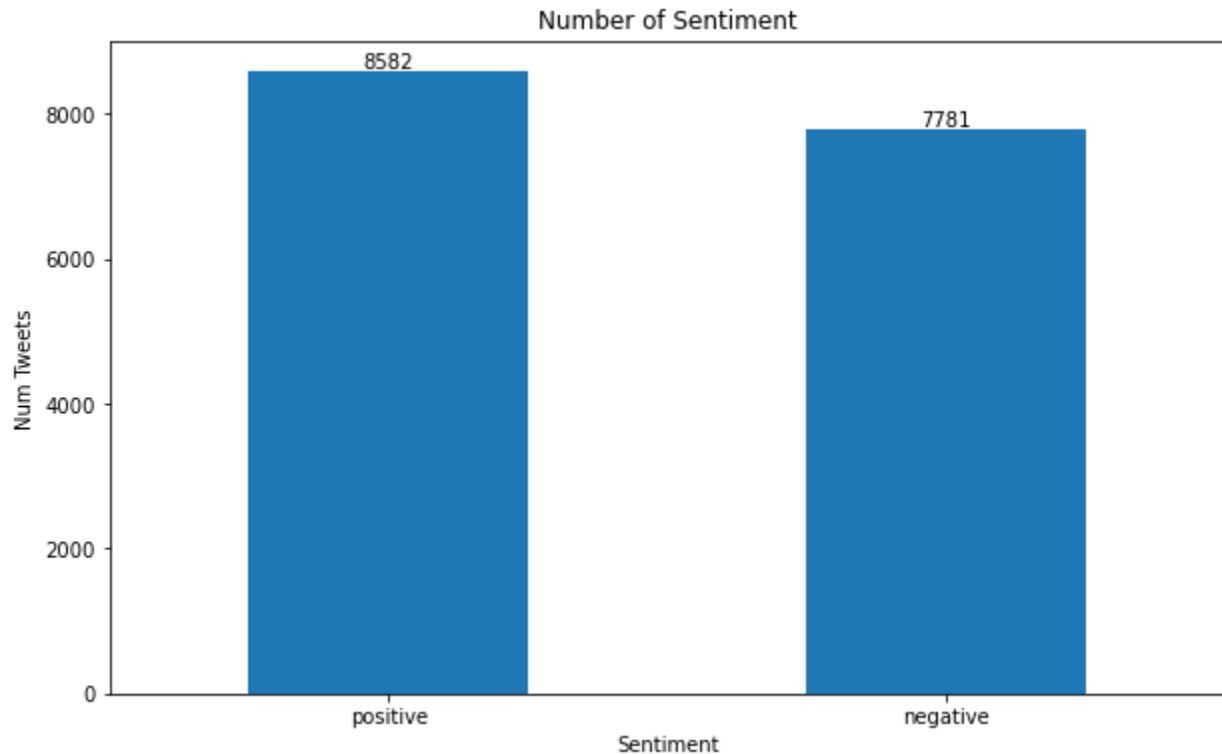


Figure 1: data after cleaning.

Embeddings

In general, word embeddings are learned based on co-occurrences of words in the sentences. The methods, such as word2vec, uses a single vector for each unique word in its vocabulary regardless of the context. The model we chose to use for this project is sentence-transformers' paraphrase-distilroberta-base-v2. This model maps input text to 768 dimensional dense vectors, and is trained on paraphrase data, which in other words means that it is trained to map texts with the same or similar meanings to similar vectors [4,5,6]. Based on this, we reasoned that this model would be a good tool to use to generate embeddings for our data as it is a reasonable assumption that Tweets expressing the same or similar emotions can be paraphrased to each other, especially given the fact that all Tweets have a strict character limit.

Models

Logistic Regression

Logistic regression is a widely used machine learning tool for statistical classification. It is the basis of more complex machine learning models, such as neural networks [1]. Logistic

Regression is a discriminative model, which aims to distinguish the classes by statistical learning. Specifically, the model takes features of the content as input and generates probability of each class that the content may belong to. For example, if given a sequence of data X and two possible class $y \in \{0, 1\}$, the logistic regression calculates $P(y = 1|X)$. Then, we can calculate the probability of the other class by computing $1 - P(y = 1|X)$.

For sentiment analysis by representing input text with word embeddings. The model then learns a vector of **weights** and a **bias term** from the labeled features in the training set. The weight vector should have the same length of the feature vector, given each feature x_i a weight w_i . Then, we gain a weighted sum for each word by multiplying its weight vector and feature vector plus the bias term. This weighted sum can be expressed as a single number $z = w \cdot x + b$. In our case, each tweet has a vector of BERT embeddings of length 768.

Logistic regression binary classification uses a sigmoid function to transform the weighted sum of features into a probability. The sigmoid function is expressed as

$$\text{sigmoid}(z) = \frac{1}{1 + \exp(-z)}. \text{ Thus, we have } P(y = 1) = \text{sigmoid}(w \cdot x + b).$$

To minimize the distance between the model output with the gold labels, an optimization algorithm is used to minimize a cross entropy loss function. After simplification, the update of weight for each time step becomes $w_{t+1} = w_t - \eta(\bar{y} - y) \cdot x_t$, where η is a learning rate that we define. The biased term can also be learned by attaching it to the end of the weight vector.

For our project, we used Scikit-learn's Logistic Regression implementation from the sklearn.linearmodel package. The hyperparameters that we tuned for this model are the solver, which is the algorithm that is used during the optimization process, as well as the penalty, which is the norm used for regularization. Furthermore, we increased the max_iter parameter, which sets the maximum number of iterations before the solver converges, to ensure that each of our models would converge [7].

Neural Net

Neural Networks are commonly used for classification, language modeling, and sequence prediction. For this project, we used Keras to build Feedforward Neural Networks as we have both in class and for homework assignments.

In this project, we use a feedforward neural network as a sentiment classifier. The model takes a list of 768 dimension embedding vectors as inputs and outputs the probability of sentiment being 1 which represents 'positive.' The feedforward neural network is built on logistic regression by adding an activation function and a hidden layer before the output. In our case, the input is slightly different from the figure, where our input is one vector of a 768 dimension embedding and output is a number between 0 and 1, representing the probability of the input being negative (0) or positive (1).

We experiment with the number of nodes in the hidden layer and activation functions of the hidden layer. We used two dense layers as the hidden layers. The first layer is fixed with 256 nodes. The hidden layer has either 128 nodes or 64 nodes, as shown in the figure below.

Layer (type)	Output Shape	Layer (type)	Output Shape
input (Dense)	(None, 256)	input (Dense)	(None, 256)
hidden (Dense)	(None, 128)	hidden (Dense)	(None, 64)
output (Dense)	(None, 1)	output (Dense)	(None, 1)

The activation function for the first hidden layer, labeled as input, is the sigmoid function. We experiment with different activation functions on the second hidden layer. The activation functions we have tried are sigmoid function, softmax function, and ReLu. We have discussed sigmoid function in the previous section. The softmax function is written as

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

ReLU function is $\text{ReLU}(x) = \max(x, 0)$.

Point of our project is to compare and contrast two different algorithms/models on the same classification task, so we chose to keep our decisions here simple - our first implementation of the neural network, based on our code from HW4, performed well on this task already, so we did some basic testing to see if we could improve performance.

Results

The results of our models can be seen in the tables below. To ensure that we were comparing the best performing versions of each algorithm, we experimented with hyper parameters as described above.

Linear Regression Results					
Hyperparameters		Metrics			
Solver	Penalty	Accuracy	Precision	Recall	F1
lbfgs	l2	0.8878	0.8988	0.8867	0.8927
lbfgs	none	0.8825	0.8956	0.8793	0.8874
newton-cg	l2	0.8878	0.8988	0.8867	0.8927
newton-cg	none	0.8827	0.8959	0.8793	0.8875
liblinear	l2	0.8878	0.8982	0.8874	0.8928
liblinear	l1	0.8900	0.9018	0.8878	0.8947
sag	l2	0.8878	0.8988	0.8867	0.8927
sag	none	0.8827	0.8959	0.8793	0.8875
saga	l2	0.8878	0.8988	0.8867	0.8927
saga	l1	0.8900	0.9017	0.8878	0.8947
saga	none	0.8829	0.8960	0.8797	0.8858

Neural Network Results					
Hyperparameters		Results			
No. of Nodes in Hidden Layer	Hidden Layer Activation Function	Accuracy	Precision	Recall	F1
128	ReLU	0.8821	0.9466	0.8224	0.8801
128	Sigmoid	0.8890	0.8911	0.8990	0.8951
128	Softmax	0.8570	0.8430	0.8952	0.8683
64	ReLU	0.8884	0.8851	0.9056	0.8952
64	<i>Sigmoid</i>	<i>0.8910</i>	<i>0.8976</i>	<i>0.8952</i>	<i>0.8964</i>
64	Softmax	0.8859	0.9396	0.8371	0.8854

As can be seen in this table, the best results for Logistic Regression came from the combination of liblinear solver and l1 penalty, resulting in 0.8900 accuracy, 0.9018 precision, 0.8878 recall, and 0.8947 F1 score.. The best results for the Neural Network came from the version with 64 nodes in the hidden layer and the sigmoid function as the activation for the hidden layer, resulting in accuracy, precision, recall, and F1 score of 0.8910, 0.8976, 0.8952, and 0.8964 respectively.

Conclusions and Future Work

There is just a small difference between the performance of the two models. The best performing model is the feedforward neural network with 64 nodes in the second hidden layer and with sigmoid activation function. It has a less than 1% better performance than the best performing logistic regression model, which is with liblinear solver and l1 penalty, on accuracy, recall, and F1, and performs marginally worse than the Linear Regression model on precision..

We suspect that the reason behind the small difference between the performance is because that logistic regression is basically just a sigmoid function. Meanwhile, the best performing feedforward neural network had a sigmoid function applied to all of its layers.

Our model generally performs very well, indicating the dataset is a good training data. All data cleaning and transformation are consistent across all subsets of data (train, validation, test). We only did basic transformations of our data, e.g. dropping columns and rows, in addition to adding a column of transformed embeddings.

In the future, we want to expand on this work by first comparing the results of using different models to generate sentence vectors for each Tweet, such as static embeddings and BERTweet. Comparing BERT with general embeddings would be a good strategy to measure how much contextual embeddings perform better than static ones. We would expect that the fine-tuned BERTweet performs better than the base sentence BERT. However, this improvement may not be significant as BERT compared to stochastic embeddings. Additionally, we want to

expand the selection of models that we compare to include ones that are less similar to each other than Logistic Regression and Neural Nets, such as Decision Trees and K-Nearest Neighbors.

We would also like to expand our dataset, or potentially find a new dataset, that has more features than just the embeddings for each tweet and their sentiment and repeat our experiments with the augmented dataset, in addition to including more nuanced sentiments than simply “positive” and “negative”.

Works Cited

- [1] “Speech and Language Processing.” <https://web.stanford.edu/~jurafsky/slp3/> (accessed Dec. 06, 2022).
- [2] “Twitter Revenue and Usage Statistics (2022),” *Business of Apps*, Oct. 05, 2018. <https://www.businessofapps.com/data/twitter-statistics/> (accessed Dec. 07, 2022).
- [3] “Twitter Tweets Sentiment Dataset.” <https://www.kaggle.com/datasets/yasserh/twitter-tweets-sentiment-dataset> (accessed Dec. 07, 2022).
- [4] “sentence-transformers/paraphrase-distilroberta-base-v2 · Hugging Face.” <https://huggingface.co/sentence-transformers/paraphrase-distilroberta-base-v2> (accessed Dec. 07, 2022).
- [5] “Paraphrase Data — Sentence-Transformers documentation.” <https://www.sbert.net/examples/training/paraphrases/README.html> (accessed Dec. 07, 2022).
- [6] “Paraphrase Mining — Sentence-Transformers documentation.” <https://www.sbert.net/examples/applications/paraphrase-mining/README.html> (accessed Dec. 07, 2022).
- [7] “sklearn.linear_model.LogisticRegression — scikit-learn 0.16.1 documentation.” https://scikit-learn.org/0.16/modules/generated/sklearn.linear_model.LogisticRegression.html (accessed Dec. 07, 2022).
- [8] S. Kotamraju, “An Intuitive Explanation of Sentence-BERT,” *Medium*, Jun. 23, 2022. <https://towardsdatascience.com/an-intuitive-explanation-of-sentence-bert-1984d144a868> (accessed Dec. 07, 2022).

Appendix

Sentiment tweets of the original dataset.

